
Smart Traffic Analyzer

Documentación Técnica

Sistema de análisis de tráfico con visión artificial,
detección de vehículos, estimación de velocidad y lectura de señales

YOLOv8n

ByteTrack

EasyOCR

OpenCV

Streamlit

PyTorch 2.11

Python 3.14

Gonzalo Solans del Carre Díaz-Gálvez · Mayo 2026

Índice de contenidos

- 1. Descripción general del sistema
- 2. Arquitectura del pipeline — 5 etapas
- 3. Tecnologías y librerías utilizadas
- 4. Detección de vehículos: YOLOv8 y filtros
- 5. Detección de señales de tráfico
- 6. Problemas encontrados y soluciones aplicadas
- 7. Entrenamiento del modelo propio
- 8. Comparativa: modelo genérico vs. fine-tuned
- 9. Conclusiones y trabajo futuro

1. Descripción general del sistema

Smart Traffic Analyzer es una aplicación web de análisis de tráfico construida sobre visión artificial. Acepta imágenes estáticas, vídeos grabados o flujo de cámara en directo y produce:

Detección y clasificación de objetos: coches, camiones, autobuses, motos, bicicletas y peatones.

Estimación de velocidad individual de cada vehículo (disponible en modo vídeo y cámara).

Cálculo de densidad de tráfico y nivel de congestión: Fluido / Moderado / Denso / Colapso.

Detección automática de señales de límite de velocidad y carteles de dirección.

Mapa de calor acumulado de concentración de vehículos.

Línea de conteo virtual con dirección (arriba / abajo).

Generación de informes PDF y exportación CSV por detección.

Vista de análisis interno: muestra todos los filtros intermedios del pipeline.

Especificaciones técnicas

Parámetro	Valor
Entrada soportada	Imagen (JPG/PNG/BMP), Vídeo (MP4/AVI/MOV/MKV), Cámara IP/USB
Clases detectadas	person · bicycle · car · motorcycle · bus · truck (6 clases)
Modelos de detección	YOLOv8n genérico + fine-tuned propio (models/best.pt)
Tracker de objetos	ByteTrack integrado en Ultralytics — IDs únicos por vehículo
OCR	EasyOCR v1.x — lectura de texto en señales y carteles

Parámetro	Valor
Lenguaje / framework	Python 3.14 · Streamlit · OpenCV · PyTorch 2.11

2. Arquitectura del pipeline — 5 etapas

El sistema se organiza en cinco etapas independientes y encadenadas, desde la adquisición de los datos de entrenamiento hasta la aplicación web final.

Etapas	Nombre	Script	Descripción
1	Adquisición de datos	download_coco_subset.py	Descarga 2.000 imágenes COCO con vehículos
2	Preprocesado y aumentación	augmentation_pipeline.py	Normalización, recorte, flip, HSV jitter
3	Entrenamiento del modelo	train.py	Fine-tuning YOLOv8n, 30 épocas
4	Inferencia y evaluación	inference.py	Predicción + ByteTrack + métricas mAP
5	Aplicación web	app.py (Streamlit)	Interfaz completa imagen/vídeo/cámara

El flujo de datos en la etapa final sigue la cadena: Imagen/Vídeo/Cámara → YOLOv8 (detección) → ByteTrack (tracking) → OpenCV (visualización) → Streamlit (interfaz).

3. Tecnologías y librerías utilizadas

YOLOv8n (Ultralytics 8.4)

Arquitectura CSPDarknet + PAN + YOLO head con 3,15 M de parámetros en la versión genérica y 3,01 M en la fine-tuned. La entrada es de 640×640 px normalizados a [0, 1] y la salida produce bounding boxes, clase y puntuación de confianza. Lleva integrado ByteTrack para asignar IDs únicos a cada vehículo. La inferencia en CPU ronda los 80 ms por imagen.

EasyOCR

Versión 1.x en modo CPU, con español e inglés cargados. Se usa exclusivamente para leer números en señales de velocidad (allowlist: '0123456789'). El umbral de confianza para confirmar una lectura es 0,50. El preprocesado aplica CLAHE y escala la imagen a un mínimo de 90 px para mejorar la legibilidad.

OpenCV (cv2)

Gestiona la mayor parte del procesado clásico de imagen: conversión BGR↔RGB↔HSV, Hough Circle Transform (param2=18), operaciones morfológicas (CLOSE + OPEN) para limpiar máscaras, GaussianBlur para suavizado, findContours + minEnclosingCircle para detección de formas y la generación del mapa de calor mediante COLORMAP_JET.

Streamlit

Interfaz web con cuatro pestañas (Imagen / Vídeo / Cámara / Resultados) construida sin HTML ni CSS manual. Usa @st.cache_resource para no recargar los modelos en cada petición, Plotly para gráficos interactivos (gauge, bar, pie), st.empty() para la actualización frame a frame y botones de descarga para PDF, CSV y JPG.

PyTorch 2.11

Backend de inferencia de Ultralytics. El entrenamiento se realizó en CPU con AMP (Automatic Mixed Precision) activado. Si hay aceleración por hardware disponible, el sistema la detecta automáticamente en ejecuciones futuras.

Otras librerías

matplotlib: gráficos en los informes PDF. pandas: tablas de detecciones y CSV. numpy: operaciones matriciales en máscaras. collections.deque: buffer circular para la

estimación de velocidad. tempfile: archivos temporales para YOLO. tqdm: barras de progreso en la descarga de COCO.

4. Detección de vehículos: YOLOv8 y filtros

Problema crítico: mapeo erróneo de IDs COCO

YOLOv8 usa los 80 IDs originales de COCO, pero el código inicial definía CLASS_NAMES como un array de 6 elementos. Esto provocaba dos fallos: COCO truck tiene ID 7, que al superar el índice 5 devolvía 'unknown' y nunca se detectaba; COCO bus tiene ID 5 y se etiquetaba incorrectamente como truck.

Solución: diccionario explícito COCO_TO_NAME = {0:'person', 1:'bicycle', 2:'car', 3:'motorcycle', 5:'bus', 7:'truck'}, obteniendo la clase con name = COCO_TO_NAME.get(cls_id).

Segundo pase para camiones y buses

Los camiones fotografiados desde atrás tienen poca similitud con las imágenes de entrenamiento de COCO, mayoritariamente laterales o frontales. Para capturarlos se añade un segundo pase con umbral reducido ($\text{truck_conf} = \max(0.18, \text{conf} \times 0.50)$, clases restringidas a bus y truck) y varios filtros anti-falso-positivo: tamaño mínimo del 2,5 % del área de imagen y $\text{IoU} < 0,30$ con detecciones ya confirmadas. Además, los bounding boxes de tipo 'car' con área $> 4 \%$ y aspecto 0,55-1,80 se reclasifican como truck (vista trasera cuadrada).

Estimación de velocidad con corrección de perspectiva

Una escala global única subestima la velocidad de vehículos lejanos. La solución aplica una escala por perspectiva por vehículo: $\text{scale} = \text{altura_real_m} / \text{altura_bbox_px}$, usando alturas de referencia de 1,5 m (car), 3,5 m (truck) y 3,2 m (bus). Así un coche a 100 m con bbox de 15 px obtiene $\text{scale} = 0,10 \text{ m/px}$, mientras que uno a 30 m con bbox de 50 px obtiene $0,03 \text{ m/px}$.

No se muestra velocidad si la altura del bbox es inferior al 5 % del frame, si el desplazamiento total es menor de 12 px o si el vehículo lleva menos de 4 frames rastreados.

Dicho esto, la velocidad que se da no es exacta ya que depende mucho de la perspectiva de la cámara y dirección de los coches, el cálculo es genérico ya que el modelo está entrenado para diferentes perspectivas de video. Para un cálculo más concreto habría que entrenarlo con la cámara siempre en el mismo sitio y los coches siempre en la misma dirección.

Heatmap y línea de conteo

El heatmap acumulativo se genera dibujando un círculo relleno centrado en cada detección, aplicando suavizado gaussiano ($\sigma=15$) y mezclando con la imagen original ($\alpha=0,45$) mediante COLORMAP_JET. La línea de conteo virtual registra cruzamientos arriba/abajo comprobando si el track anterior y el actual están en lados opuestos de la línea, contando cada ID solo una vez gracias a un conjunto `_crossed`.

Cálculo de densidad y nivel de congestión

$\text{density} = \Sigma(\text{área de cada bounding box}) / \text{área total de imagen} \times 100$. Niveles: Fluido (0-20 %), Moderado (20-50 %), Denso (50-75 %), Colapso (>75 %). La densidad refleja el espacio visual ocupado, no el número absoluto de vehículos.

5. Detección de señales de tráfico

Pipeline: señal de límite de velocidad

El detector de señales de velocidad aplica ocho pasos en secuencia. Primero calcula una máscara HSV roja amplia ($H[0,20][155,180]$, $S \geq 50$, $V \geq 40$) para cubrir condiciones de atardecer y nocturnas. A continuación, aplica morfología (CLOSE $k=7$ iter=2 + OPEN $k=7$ iter=1) para cerrar huecos en el anillo rojo. Extrae hasta 25 contornos por tamaño decreciente y filtra por aspect ratio (0,45-2,20) y tamaño mínimo (2,5 % de la imagen). Evita duplicados si dos detecciones están a menos de $1,3 \times \text{radio}$. Recorta el interior con $\text{shrink} = \max(r \times 0,22, 6 \text{ px})$ y ejecuta EasyOCR con allowlist '0123456789' y umbral de confianza 0,50. Finalmente, elimina lecturas parciales ('10' si también existe '100').

Pipeline: carteles de dirección (azul/verde)

La máscara azul usa $H[95,135]$, $S \geq 80$, $V \geq 60$; la verde, $H[42,88]$, $S \geq 60$, $V \geq 50$ con la restricción verde AND NOT azul para evitar solapamientos. Tras morfología CLOSE+OPEN, se buscan contornos con aspect entre 0,40 y 7,0, ancho mínimo de 60 px y solidez $\geq 0,45$. El OCR completo (sin allowlist) requiere confianza $\geq 0,50$ para texto de alta confianza y $\geq 0,25$ para baja. Si no se detecta ningún texto, la región se descarta.

Filtro _is_ring: discriminador geométrico de corona

Una señal de velocidad tiene estructura de corona: borde rojo exterior e interior blanco. El filtro calcula $r_{inner} = \text{int}(r \times 0,68)$ y comprueba que al menos el 20 % de los píxeles de la corona exterior sean rojos, y que el interior no supere el 18 % de rojo (para descartar faros sólidos y arcos de túnel).

Filtro _sign_inside_vehicle

El IoU clásico es insuficiente cuando la señal (80×80 px) está dentro de un camión (400×300 px): el IoU ronda el 5 % y no se suprime. La solución comprueba si el centro de la señal está dentro del bounding box del vehículo, o si la intersección cubre más del 40 % del área de la señal. Se aplica a señales regulatorias y carteles antes de combinar los resultados finales.

6. Problemas encontrados y soluciones aplicadas

Tipo	Problema	Solución
BUG CRÍTICO	Camiones nunca detectados	Diccionario COCO_TO_NAME con IDs correctos {5:bus, 7:truck}
FALSO POSITIVO	Señales de velocidad inventadas (10, 80, 100, 110)	Ordenar candidatos por radio descendente. Bloquear zona ya confirmada. Post-proceso startswith.

Tipo	Problema	Solución
FALSO POSITIVO	Stop detectado aunque no hubiera ninguno	Umbral elevado a 0,78 para stop/semáforo. Stop eliminado del detector finalmente.
FALSO POSITIVO	Camión detectado como señal de velocidad	Test <code>_sign_inside_vehicle</code> : comprueba centro y solapamiento del 40 % del área de la señal.
BUG	Velocidades muy lentas (20 km/h en lugar de 80)	Corrección de perspectiva: <code>scale = altura_real_m / bbox_height_px</code> .
BUG	Error de caché: 'no attribute detect_regulatory_signs'	Bump del parámetro <code>_version</code> en <code>get_sign_detector()</code> fuerza recrear la instancia.
BUG	<code>cv2.contourArea()</code> devuelve área del disco, no del anillo	<code>cv2.countNonZero(bitwise_and(red_mask, circle_mask)) / circle_area</code> . Umbral 0,10-0,65.
PERFORMANCE	Vehículos desaparecían tras <code>detect_standard_signs</code>	Instancia separada <code>_sign_model = YOLO('yolov8n.pt')</code> solo para señales.

7. Entrenamiento del modelo propio

Motivación

YOLOv8n preentrenado en COCO funciona bien para detección general, pero presenta limitaciones: las vistas traseras de camiones son escasas en COCO (mayoritariamente laterales o frontales), la distribución de clases está desbalanceada y el mapeo de IDs estaba erróneo. El fine-tuning parte de los pesos preentrenados y ajusta los últimos pasos del modelo para el subconjunto de 6 clases de tráfico. El resultado es el modelo `models/best.pt` que la app carga automáticamente si existe.

Pipeline de datos

La Etapa 1 descarga 2.000 imágenes del conjunto de validación COCO 2017 que contienen al menos un vehículo de las 6 clases objetivo. La Etapa 2 convierte las anotaciones COCO JSON a formato YOLO txt, aplica split 80/20 (train/val) y aumentación con $hsv_h=0,015$, $hsv_s=0,7$, $hsv_v=0,4$, $fliplr=0,5$, $scale=0,5$, $mosaic=1,0$ y $erasing=0,4$.

Configuración del entrenamiento

Parámetro	Valor
Modelo base	YOLOv8n (yolov8n.pt)
Épocas / mejor época	30 épocas, mejor en época 28 (mAP50=0,4802)
Batch size / Image size	8 / 640×640 px
Optimizer / lr0 / lrf	Auto (AdamW) / 0,01 / 0,01
Momentum / Weight decay	0,937 / 0,0005
Warmup / Patience (early stopping)	3 épocas / 10
AMP	Activado
Clases entrenadas	person · bicycle · car · motorcycle · bus · truck

8. Comparativa: modelo genérico vs. fine-tuned

Evaluación sobre el conjunto de validación (400 imágenes, 1.836 instancias). El modelo genérico se evaluó sobre el mismo dataset de 6 clases, no sobre COCO completo.

Métricas globales

Métrica	Genérico	Fine-tuned	Δ
mAP50	0,4787	0,4808	+0,21 %
mAP50-95	0,3222	0,2748	-4,74 %
Precision	0,5076	0,5848	+7,72 %
Recall	0,4338	0,4239	-0,99 %

AP50 por clase

Clase	Genérico	Fine-tuned
person	0,75	0,68
bicycle	0,48	0,22
car	0,70	0,55
motorcycle	0,88	0,60
bus	0,06	0,51
truck	0,00	0,32

Análisis e interpretación

El fine-tuning mejora la precisión (+15,2 %) a costa de un ligero descenso en recall (-2,3 %) y mAP50-95 (-4,7 %). El modelo fine-tuned es más conservador: detecta menos objetos pero con mayor confianza en los que detecta.

El resultado más llamativo es la clase bus: el modelo genérico obtiene AP50 $\approx$ 0,06 (casi cero) por el bug de mapeo de IDs; el fine-tuned alcanza 0,51. La clase truck pasa de AP=0,00 a AP50=0,32 —sigue siendo bajo porque las vistas traseras son difíciles—, pero el modelo ahora las detecta.

Limitación principal: los datos de COCO son escenas internacionales. Un fine-tuning adicional con imágenes de dashcam en autopistas españolas mejoraría la detección en el contexto real de uso.

9. Conclusiones y trabajo futuro

Logros del proyecto

Pipeline completo de visión artificial: datos → preprocesado → entrenamiento → inferencia → aplicación web.

Detección de 6 clases con corrección del bug crítico de IDs COCO.

Estimación de velocidad con corrección de perspectiva por vehículo.

Detección de señales de velocidad mediante HSV + contornos + OCR.

Lectura de carteles de dirección con EasyOCR y validación por texto.

Modelo fine-tuned propio (best.pt) que mejora bus en +753 % AP50.

Interfaz web completa con heatmap, línea de conteo e informes PDF/CSV.

Limitaciones conocidas

Señales de velocidad en condiciones nocturnas extremas o muy borrosas pueden no detectarse (el OCR falla con baja resolución).

La estimación de velocidad requiere calibración para cada cámara concreta, ya que la perspectiva varía según ángulo y altura de instalación.

Los datos de entrenamiento de COCO son escenas internacionales, con pocas vistas traseras de camiones.

El modelo no detecta motos de reparto ni furgonetas con alta fiabilidad desde ángulos poco comunes.

La detección de señales es puramente clásica (HSV + OCR), no neuronal, y es sensible a iluminación atípica.

Trabajo futuro

Coger imágenes propias de carretera española con Roboflow y reentrenar con ese dataset.

Sustituir el detector clásico de señales por un modelo YOLO especializado en señales de tráfico europeas.

Calibración automática de velocidad usando marcas del carril (3 m estándar) como referencia de escala.

Añadir detección de comportamientos: cambio de carril, distancia de seguridad.

Exportar el modelo a ONNX/TensorRT para inferencia en tiempo real en GPU.

Integración con cámara IP fija para monitorización permanente de un tramo.

Resumen

Métrica	Valor
Líneas de código aproximadas	~2.500
Ficheros Python	12
Clases detectadas	6
Bugs resueltos documentados	8+
mAP50 alcanzado (fine-tuned)	48,1 %